

Diplomarbeit

EduShelf

Verfasser: Phillip Rubak 5AHITS
Elias Welt 5AHITS

Betreuer: Dipl.-Ing. Gerald Stoll

Schuljahr: 2025/26

Abgabevermerk:

Datum: 07.04.2025

übernommen von:

Höhere Technische Bundeslehranstalt Hollabrunn

Höhere Lehranstalt für Informationstechnologie

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

Hollabrunn, am 07.04.2025, Phillip Rubak

Hollabrunn, am 07.04.2025, Elias Welt

1 HINWEISE

Die vorliegende Diplomarbeit wurde in Zusammenarbeit mit der Firma "X-Works" ausgeführt.

Die in dieser Diplomarbeit entwickelten Prototypen und Software-Produkte dürfen ganz oder auch in Teilen von Privatpersonen oder Firmen nur dann in Verkehr gebracht werden, wenn sie diese selbst geprüft und für den vorgesehenen Verwendungszweck für geeignet befunden haben.

Es wird keinerlei Haftung übernommen für irgendwelche Schäden, die aus der Nutzung der hier entwickelten oder beschriebenen Bestandteile des Projekts resultieren.

Für alle Entwicklungen gilt die GNU General Public License [<http://www.gnu.org/licenses/gpl.html>] der Free Software Foundation, Boston, USA in der Version 3.

2 SCHLÜSSELBEGRIFFE

- TeamTrackr
- Android - Entwicklung
- iOS - Entwicklung
- Backend
- Frontend
- AI

3 DANKSAGUNGEN

Wir möchten an dieser Stelle unseren aufrichtigen Dank all jenen aussprechen, die uns während der Entstehung dieser Diplomarbeit unterstützt haben.

Unser besonderer Dank gilt Tailored Apps, unserer Partnerfirma, die uns nicht nur mit wertvollen Einblicken und praxisnaher Unterstützung begleitet hat, sondern auch eine entscheidende Rolle in der erfolgreichen Umsetzung unseres Projekts gespielt hat. Die Zusammenarbeit mit euch war für uns eine wertvolle Erfahrung.

Ebenso möchten wir uns herzlich bei Herrn Geischläger, unserem Diplomarbeitsbetreuer, bedanken. Seine fachliche Expertise, seine konstruktiven Anregungen und seine stetige Unterstützung haben maßgeblich zur Qualität unserer Arbeit beigetragen.

Ein großer Dank gilt auch allen, die uns auf diesem Weg unterstützt haben – sei es durch hilfreiches Feedback, motivierende Worte oder ihr Verständnis und ihre Geduld.

Ohne diese wertvolle Unterstützung wäre unsere Diplomarbeit in dieser Form nicht möglich gewesen.

DIPLOMARBEIT DOKUMENTATION

Name der Verfasser/innen	Phillip Rubak, Elias Welt
Jahrgang Schuljahr	2025/26
Thema der Diplomarbeit	EduShelf
Kooperationspartner	—

Aufgabenstellung	Im Rahmen dieser Diplomarbeit wird die Entwicklung einer innovativen App für Teamkommunikation und Aufgabenmanagement vorgestellt. Ziel ist es, eine nahtlose Lösung bereitzustellen, die Organisationen und Teams dabei unterstützt, effizient zusammenzuarbeiten und Projekte erfolgreich zu managen. Die App integriert essenzielle Funktionen wie Authentifizierung, Kalender, Chat inklusive KI-gestütztem Chatbot sowie die Erstellung und Verwaltung von Teams. Darüber hinaus ermöglicht sie ein umfassendes Budget- und Zeitmanagement auf Projektebene. Ein zentraler Bestandteil des Projekts ist die Entwicklung nativer Anwendungen für iOS, Android, Desktop und Web sowie eines leistungsfähigen Backends. Das Backend gewährleistet die Verwaltung aller Frontend-Anforderungen, speichert Daten in einer zentralen Datenbank und verarbeitet Benutzeranfragen effizient. Durch die Integration aller Funktionen in einer benutzerfreundlichen Plattform wird die Fragmentierung bestehender Tools überwunden und eine reibungslose Zusammenarbeit gefördert. Das geplante Endprodukt stellt eine umfassende Lösung für modernes Projektmanagement dar, die die Bedürfnisse moderner Teams adressiert und die Effizienz sowie den Erfolg in Organisationen steigert.
------------------	---

Realisierung	Die Anliegen der Themenstellungen umfassen die Entwicklung einer iOS / Android / Desktop / Web App sowie eines Backends. Die nativen Applikationen sollen folgende Kernfeatures beinhalten Login und Registrierung (Authentifizierung), Kalender, eine Chat-Möglichkeit inklusive AI unterstützten Chatbot, sowie die Möglichkeit zur Erstellung und Verwaltung von Teams, in diesen soll es die Opportunität zur Budgetverwaltung und Zeitverwaltung von einzelnen Projekten geben. Die Webapplikation soll die selben bzw. ähnlich implementierten Features haben. Das Backend soll in der Lage sein alle Anforderungen der jeweiligen Frontends zu verwalten sowie in der Datenbank zu speichern und deren Abrufe zu verarbeiten.
Ergebnisse	Das geplante Ergebnis umfasst die fertigen iOS-, Android-, Desktop- und Web-Anwendungen sowie ein Backend. Die nativen Applikationen beinhalten folgende Kernfunktionen: Benutzer-Login und -Registrierung (Authentifizierung), einen Kalender, eine Chat-Funktion mit einem KI-gestützten Chatbot sowie die Möglichkeit zur Erstellung und Verwaltung von Teams und deren Projekten. Innerhalb der Teams werden Funktionen für Budget- und Zeitmanagement bereitgestellt. Die Webanwendung hat ähnliche oder identische Funktionen. Das Backend ist so entwickelt, dass es die Anforderungen der jeweiligen Frontends erfüllt, Daten in einer Datenbank speichert und diese für verschiedene Anfragen der Applikationen verfügbar macht. Des Weiteren gewährleisten die Anwendungen eine benutzerfreundliche Bedienung.
Typische Grafik, Foto etc. (mit Erläuterung)	MISSING SVG: img/blockschaltbildSVG.svg
Teilnahme an Wettbewerben, Auszeichnungen	
Möglichkeiten der Einsichtnahme in die Arbeit	HTL Hollabrunn Anton Ehrenfriedstraße 10 2020 Hollabrunn

Approbation (Datum / Unterschrift)	Prüfer/Prüferin	Direktor/Direktorin Abteilungsvorstand/ Abteilungsvorständin
---------------------------------------	-----------------	--

	HÖHERE TECHNISCHE BUNDESLEHRANSTALT HOLLABRUNN COLLEGE of ENGINEERING
	Information Technology

DIPLOMA THESIS

Documentation

Author(s)	Phillip Rubak, Elias Welt,
From Academic year	2025/26
Topic	EduShelf
Co-operation partners	TailoredApps

Task Description	This diploma project focuses on developing an innovative app for team communication and task management. The goal is to provide a seamless solution that helps organizations and teams collaborate efficiently and successfully manage projects. The app integrates essential features such as authentication, a calendar, chat including an AI-powered chatbot, and team creation and management. Additionally, it offers comprehensive budget and time management at the project level. A key aspect of the project is the development of native applications for iOS, Android, Desktop, and Web, along with a powerful backend. The backend handles all frontend requests, stores data in a central database, and processes user interactions efficiently. By integrating all functions into a user-friendly platform, the fragmentation of existing tools is reduced, fostering smooth collaboration. The final product will provide a comprehensive solution for modern project management, addressing the needs of contemporary teams while enhancing efficiency and success within organizations.
Implementation	The project involves developing a mobile and desktop application for iOS, Android, Desktop, and Web, as well as a backend system. The native applications will include core features such as login and registration (authentication), a calendar, chat functionality with AI support, and the ability to create and manage teams. Within these teams, users will have options for budget and time management at the project level. The web application will offer similar features. The backend will be designed to manage all frontend requirements, store data in a database, and process requests efficiently.
Results	The expected outcome includes fully developed iOS, Android, Desktop, and Web applications, along with a backend system. The native applications will feature user login and registration (authentication), a calendar, chat functionality with an AI-powered chatbot, and the ability to create and manage teams and projects. Teams will have access to budget and time management tools. The web application will provide similar or identical functionality. The backend will be designed to support all frontend applications, store data in a database, and efficiently process requests. Additionally, the applications will ensure a user-friendly experience.
Typische Grafik, Foto etc. (mit Erläuterung)	MISSING SVG: img/blockschaltbildSVG.svg

Participation in competitions Awards		
Accessibility of final project thesis	HTL Hollabrunn Anton Ehrenfriedstraße 10 2020 Hollabrunn	
Approval (Date / Signature)	Examiner/s	Head of Department / College

Thema

TeamTrackr

Verlauf

- 13.06.2024: Das Thema 'TeamTrackr' wurde durch Franz Geischläger angelegt.
- 13.06.2024: Sie wurden zum Thema hinzugefügt.
- 13.06.2024: Sie wurden zum Thema hinzugefügt.
- 13.06.2024: Sie wurden zum Thema hinzugefügt.
- 16.09.2024: Thema wurde eingereicht.
- 16.09.2024: Thema wurde von Franz Geischläger akzeptiert.
- 16.09.2024: Thema wurde von Wilfried Trollmann akzeptiert.
- 23.09.2024: Thema wurde von Wolfgang Bodei akzeptiert.

Schulische Informationen

- **Schulart:** Technische, gewerbliche und kunstgewerbliche Schulen
- **Klasse:** 5AHITS
- **Abteilung:** IT
- **Schuljahr der abschließenden Prüfung:** 2024/25

Betreuung

Rolle	Name
Hauptverantwortlicher Betreuer	Franz Geischläger (Akzeptiert)
Direktion	Wolfgang Bodei (Akzeptiert)
Abteilungsvorstand	Wilfried Trollmann (Akzeptiert)

Kooperationspartner / Auftraggeber

- **Name:** Tailored Apps
- **Adresse:** Heiligenstädterstraße 31/1, 1190 Wien
- **URL:** www.tailored-apps.com
- **E-Mail:** office@tailored-apps.com
- **Ansprechpartner:** Martin Zehetner
- **Telefon:** +43 18902845
- **Typ:** Sonstige

Beteiligte SchülerInnen

Schüler/in	Individuelle Themenstellung	Abteilung
Stefan Bayer	Android-Development	IT
Kurt Broneder	Web-Development	IT
Suad Demiri	iOS-Development	IT
Lorenz Geyser	Backend-Development	IT

Ausgangslage

Derzeit existiert keine Applikation, die es ermöglicht, Organisationen, Projekte und deren Teams effizient und benutzerfreundlich zu managen. Darüber hinaus fehlt eine Lösung, die alle essenziellen Tools integriert, um ein solches effizientes und benutzerfreundliches Management zu gewährleisten.

Untersuchungsanliegen der individuellen Themenstellungen

Die Anliegen der Themenstellungen umfassen die Entwicklung einer iOS-, Android-, Desktop- und Web-App sowie eines Backends. Die nativen Applikationen sollen folgende Kernfeatures beinhalten:

- Login und Registrierung (Authentifizierung)
- Kalender
- Eine Chat-Möglichkeit inklusive AI-unterstütztem Chatbot
- Erstellung und Verwaltung von Teams
- Budgetverwaltung und Zeitverwaltung von Projekten

Die Webapplikation soll dieselben bzw. ähnlich implementierten Features haben. Das Backend soll alle Anforderungen der jeweiligen Frontends verwalten, Daten in einer Datenbank speichern und Anfragen verarbeiten.

Zielsetzung

Unsere App bietet eine nahtlose Lösung für Teamkommunikation und Aufgabenmanagement. Mit leistungsstarken Funktionen unterstützt sie Teams und Organisationen dabei, effizient zusammenzuarbeiten und ihre Projekte erfolgreich zu verfolgen. Durch die Integration von Kommunikation und Projektverfolgung löst unsere App das Problem der Fragmentierung und fördert eine reibungslose Zusammenarbeit.

Geplantes Ergebnis der individuellen Themenstellungen

Das geplante Ergebnis umfasst die fertigen iOS-, Android-, Desktop- und Web-Anwendungen sowie ein Backend. Die nativen Applikationen beinhalten folgende Kernfunktionen:

- Benutzer-Login und -Registrierung (Authentifizierung)
- Kalender
- Chat-Funktion mit einem KI-gestützten Chatbot
- Erstellung und Verwaltung von Teams und deren Projekten
- Budget- und Zeitmanagement innerhalb der Teams

Die Webanwendung wird ähnliche oder identische Funktionen haben. Das Backend wird so entwickelt, dass es die Anforderungen der jeweiligen Frontends erfüllt, Daten in einer Datenbank speichert und diese für verschiedene Anfragen der Applikationen verfügbar macht. Des Weiteren gewährleisten die Anwendungen eine benutzerfreundliche Bedienung.

Meilensteine

Meilenstein	Datum
Fertigstellung Grundkonzept	20.06.2024
Fertigstellung Backend	22.11.2024
Fertigstellung Frontend	20.12.2024
Fertigstellung Applikationen	24.01.2025
Fertigstellung Dokumentation	14.03.2025

Erklärung

Die unterfertigten Kandidaten/Kandidatinnen haben gemäß den geltenden schulrechtlichen Bestimmungen die Ausarbeitung einer abschließenden Arbeit (Diplomarbeit bzw. Abschlussarbeit) mit folgender Aufgabenstellung gewählt:

TeamTrackr

Individuelle Aufgabenstellungen im Rahmen des Gesamtprojektes:

- Lorenz Geyser (4AHITS): **Backend-Development**
- Suad Demiri (4AHITS): **iOS-Development**
- Kurt Broneder (4AHITS): **Web-Development**
- Stefan Bayer (4AHITS): **Android-Development**

Die Kandidaten/Kandidatinnen nehmen zur Kenntnis, dass die abschließende Arbeit in eigenständiger Weise und außerhalb des Unterrichtes zu bearbeiten und anzufertigen ist, wobei Ergebnisse des Unterrichtes mit einbezogen werden können, die jedenfalls als solche entsprechend kenntlich zu machen sind.

Die Abgabe der vollständigen abschließenden Arbeit hat in digitaler und in zweifach ausgedruckter Form bis spätestens **07.04.2025** beim zuständigen Betreuer/der zuständigen Betreuerin zu erfolgen.

Die Kandidaten/Kandidatinnen nehmen auch zur Kenntnis, dass ein Abbruch der abschließenden Arbeit nicht möglich ist.

Kandidaten/Kandidatinnen:

Datum und Unterschrift bzw. Handysignatur:

Lorenz Geyser (4AHITS)

Lorenz Geyser 13.9.2025

Suad Demiri (4AHITS)

Suad Demiri 13.9.2025

Kurt Broneder (4AHITS)

Kurt Broneder 13.9.2025

Stefan Bayer (4AHITS)

Stefan Bayer 13.9.2025

Inhaltsverzeichnis

1	HINWEISE	5
2	SCHLÜSSELBEGRIFFE	6
3	DANKSAGUNGEN	7
3.1	Motivation für die Arbeit	21
3.2	Umfeld und Stand der Technik	21
3.3	Aufgabenstellung	21
3.3.1	Ziele	22
4	Frontend Implementierung	24
4.1	Einführung und Zielsetzung	24
4.2	Technologie-Stack	25
4.2.1	React (v19)	25
4.2.2	Vite	25
4.2.3	React Router (v7)	25
4.3	Projektsetup und Konfiguration	26
4.4	Softwarearchitektur und Datenfluss	27
4.4.1	Unidirektionaler Datenfluss	27
4.4.2	Routing-Strategie	27
4.5	Datenschicht und API-Integration	28
4.5.1	API-Wrapper	28
4.6	Implementierung der Kernkomponenten	29
4.6.1	Authentifizierungssystem	29
4.6.2	Dateimanagement (Files.jsx)	29
4.6.3	Interaktives Lernen: Das Quiz-Modul	30

4.6.3.1	Datenstruktur	30
4.6.3.2	Ablauf eines Quiz	30
4.6.4	Flashcards (Lernkarten)	30
4.6.5	Chat und KI-Integration	31
4.6.6	Admin Panel und RBAC	31
4.7	Benutzeroberfläche und Design (UI/UX)	32
4.7.1	Styling-Strategie	32
4.7.2	Dark Mode Implementierung	32
4.8	Detaillierte Analyse ausgewählter Module	33
4.8.1	Quiz-Editor Logik (QuizModal.jsx)	33
4.8.1.1	Zustandsverwaltung	33
4.8.1.2	Dynamische Formular-Manipulation	33
4.8.1.3	Validierung und Speichern	34
4.8.2	Flashcards: 3D-Interaktion und State	34
4.8.3	Admin Panel: Kaskadierende Datenabfragen	34
4.9	Deployment und Betrieb	35
4.9.1	Docker-Containerisierung	35
4.9.2	Nginx Konfiguration	36
4.10	Herausforderungen und Lösungen	36
4.10.1	Synchronisation des UI-Status	36
4.10.2	Performance bei großen Dateilisten	37
4.11	Ausblick und Erweiterungsmöglichkeiten	37
4.12	Fazit	37
5	Backend Implementierung	40
5.1	Systemarchitektur	40
5.1.1	Presentation Layer (Controllers)	40
5.1.2	Business Logic Layer (Services)	41
5.1.3	Data Access Layer (Data)	41
5.1.4	Infrastructure Layer	42
5.2	Technologie-Stack	42
5.3	Projekt-Setup und Konfiguration	43
5.3.1	Programm-Initialisierung (Program.cs)	43

5.3.2	Konfiguration (appsettings.json)	44
5.4	Datenmodelle und Datenbankdesign	44
5.4.1	Kernelemente	44
5.5	API Design und Controller	45
5.5.1	Wichtige Endpunkte	45
5.6	Sicherheitskonzepte	46
5.6.1	Authentifizierung & Autorisierung	46
5.6.2	Datenvalidierung	46
5.6.3	Error Handling	47
5.7	Ingestion-Pipeline (Datenverarbeitung)	47
5.7.1	1. Textextraktion (OCR & Parsing)	47
5.7.2	2. Chunking (Intelligente Segmentierung)	48
5.7.3	3. Embedding und Vektorspeicherung	48
5.8	Detaillierte Analyse ausgewählter Module	49
5.8.1	Authentifizierung: Secure Cookies und Hashing	49
5.8.1.1	Passwort-Hashing (BCrypt)	49
5.8.1.2	Cookie-Konstruktion	49
5.8.2	Ingestion-Pipeline: HNSW Indexierung	50
5.9	RAG (Retrieval-Augmented Generation) Workflow	50
5.9.1	Retrieval-Strategie	50
5.9.2	Prompt Engineering und Kontext-Management	51
5.10	Infrastruktur & Deployment	51
5.10.1	Docker-Komposition	51
5.10.1.1	Datenbank (PostgreSQL mit pgvector)	51
5.10.1.2	KI-Inferenz (Ollama)	52
5.10.1.3	Object Storage (MinIO)	52
5.10.2	Dockerfile	53
5.11	Herausforderungen und Lösungen	53
5.11.1	Blockierende Operationen bei großen Dateien	53
5.11.2	LLM Kontext-Fenster-Limits	54
5.11.3	Datenkonsistenz bei Löschoperationen	54
5.12	Qualitätssicherung und Tests	54
5.12.1	Unit Tests (xUnit)	54

5.12.2 Integration Tests	55
5.13 Fazit	55

3.1 Motivation für die Arbeit

Die effektive Integration von Management-Instrumentarien für Organisationsstrukturen, Projektkoordination und Teamführung stellt ein signifikantes Desiderat in der aktuellen digitalen Infrastruktur dar. Gegenwärtige Analysen dokumentieren eine Fragmentierung der verfügbaren Softwarelösungen, wodurch die Realisierung eines kohärenten und effizienten Organisationsmanagements substantiell limitiert wird. Die Diskrepanz zwischen vorhandenen partikulären Anwendungen und der Anforderung nach einer holistischen Managementplattform manifestiert sich in beträchtlichen Effizienzdefiziten sowie suboptimaler Ressourcenallokation innerhalb organisatorischer Einheiten. Diese Forschungsarbeit adressiert die identifizierte Problematik mittels einer systematischen Konzeption und Implementation einer integrierten Softwarelösung zur Überwindung existierender technologischer Limitationen.

3.2 Umfeld und Stand der Technik

Im Kontext der gegenwärtigen technologischen Entwicklung existieren divergente Einzelsysteme zur Adressierung spezifischer Management-Aspekte. Eine kritische Evaluation der Fachliteratur indiziert signifikante Korrelationen zwischen fragmentierten Management-Infrastrukturen und reduzierten Produktivitätsparametern sowie verminderter Nutzerakzeptanz. Die Extrapolation aktueller Entwicklungstendenzen im Bereich digitaler Management-Systeme lässt eine progressive Konvergenz einzelner Funktionalitäten antizipieren, jedoch fehlt bislang eine vollständig interoperable Plattform mit multimodaler Zugänglichkeit. Existierende Systeme offerieren elaborierte Funktionalitäten im Bereich der Kommunikationsfazilitation, während ein anderes System primär exzelliert im Kontext des Projektmanagements. Eine Integration dieser disparaten Ansätze in eine kohärente Systementität wurde bislang nicht realisiert. Die vorliegende Forschungsarbeit positioniert sich in dieser identifizierten Forschungslücke mit dem Ziel der Entwicklung einer konsolidierten Management-Infrastruktur.

3.3 Aufgabenstellung

Die Forschungsaufgabe umfasst die systematische Konzeption und Implementierung eines multimodalen Anwendungssystems mit nativen Applikationen

für iOS, Android und Desktop-Systeme sowie einer komplementären Web-Applikation, unterstützt durch eine zentralisierte Backend-Infrastruktur. Die primären Forschungs- und Entwicklungsziele umfassen:

- Entwicklung eines hochsicheren Authentifizierungsmechanismus unter Berücksichtigung aktueller kryptographischer Standards und Sicherheitsparadigmen
- Implementation eines semantisch strukturierten Kalendersystems mit integrierter Ressourcenallokationsfunktionalität
- Konzeption und Realisierung eines Kommunikationssystems mit bidirektionaler Informationstransmission sowie Integration eines künstlich-intelligenten Konversationssystems zur Optimierung der Kommunikations-effizienz

Die Backend-Architektur erfordert eine optimierte Datenbankstruktur mit adäquater Skalierbarkeit sowie eine performante API-Implementation zur Gewährleistung effizienter plattformübergreifender Datensynchronisation und -konsistenz.

3.3.1 Ziele

Das primäre wissenschaftliche Ziel dieser Forschungsarbeit konstituiert sich in der Entwicklung und Evaluation einer vollständig funktionalen Management-Plattform mit multimodaler Zugänglichkeit auf diversen Endgeräten (iOS, Android, Desktop, Web) unter Berücksichtigung folgender Forschungsziele:

1. Konzeption und Implementation einer optimierten Systemarchitektur mit effizienten Datenflussstrukturen zwischen Frontend- und Backend-Komponenten unter Anwendung aktueller Software-Engineering-Prinzipien
2. Realisierung einer kognitiv ergonomischen Benutzeroberfläche mit konsistenter visueller und funktionaler Repräsentation über heterogene Plattformen hinweg, basierend auf etablierten Usability-Heuristiken
3. Entwicklung eines kryptographisch robusten Authentifizierungssystems zur Gewährleistung der Datenintegrität und -vertraulichkeit

4. Implementation eines multifunktionalen Team- und Projektmanagementsystems mit präzisen Funktionalitäten zur Budgetierung und temporalen Ressourcendisposition unter Berücksichtigung aktueller Projektmanagement-Methodologien
5. Konzeption und Realisierung eines intelligenten Kommunikationssystems mit Integration von Natural Language Processing-Algorithmen zur Optimierung der Informationstransmission und Teamkoordination

Die Verifizierung der Zielerreichung erfolgt mittels systematischer empirischer Evaluation unter Anwendung etablierter quantitativer und qualitativer Forschungsmethoden, um die Konformität mit den spezifizierten Anforderungen hinsichtlich Funktionalität, Benutzerfreundlichkeit und Systemperformanz zu validieren.

4 Frontend Implementierung

4.1 Einführung und Zielsetzung

Das Frontend der Anwendung „EduShelf“ repräsentiert die Schnittstelle zwischen dem Benutzer und den komplexen Backend-Diensten. Ziel der Implementierung war die Schaffung einer modernen, performanten und intuitiven Benutzeroberfläche, die das Lernen und Verwalten von Dokumenten so effizient wie möglich gestaltet.

Im Gegensatz zu klassischen Webanwendungen, die auf dem „Multi-Page Application“ (MPA) Prinzip basieren und bei jeder Interaktion eine neue HTML-Seite vom Server anfordern, wurde EduShelf als „Single Page Application“ (SPA) konzipiert. In einer SPA wird lediglich beim ersten Aufruf eine einzige HTML-Datei (`index.html`) geladen. Alle weiteren Interaktionen finden clientseitig statt, indem JavaScript (in diesem Fall React) den DOM (Document Object Model) dynamisch manipuliert.

Die Vorteile dieses Architekturansatzes sind vielfältig:

- **Performance:** Da nach dem initialen Laden nur noch Rohdaten (im JSON-Format) und keine vollständigen HTML-Seiten mehr übertragen werden müssen, reduziert sich das Datenvolumen signifikant.
- **Benutzererfahrung (UX):** Übergänge zwischen verschiedenen Ansichten erfolgen nahezu instantan, ohne das typische Flackern eines Seiten-Reloads. Dies vermittelt das Gefühl einer nativen Desktop-Anwendung.
- **Entkopplung:** Durch die strikte Trennung von Frontend und Backend über eine REST-API können beide Systeme unabhängig voneinander entwickelt, getestet und skaliert werden.

4.2 Technologie-Stack

Die Auswahl der Technologien erfolgte unter den Gesichtspunkten der Modernität, Verbreitung (Community-Support) und Wartbarkeit.

4.2.1 React (v19)

React ist eine von Facebook (Meta) entwickelte JavaScript-Bibliothek zur Erstellung von Benutzeroberflächen. Im Kern basiert React auf Komponenten, die wiederverwendbare Bausteine der UI darstellen. Ein Schlüsselement von React ist das „Virtual DOM“. Anstatt bei jeder Änderung direkt das langsame, reale DOM des Browsers zu manipulieren, erstellt React eine virtuelle Kopie im Speicher. Bei Zustandsänderungen vergleicht React den neuen virtuellen Baum mit dem alten (Diffing-Algorithmus) und berechnet die effizienteste Methode, um das reale DOM zu aktualisieren.

In EduShelf wird ausschließlich die „Functional Component“ Syntax in Verbindung mit „Hooks“ verwendet. Hooks wie `useState` oder `useEffect` ermöglichen es, Zustand und Seiteneffekte in funktionalen Komponenten zu nutzen, ohne Klassen schreiben zu müssen. Dies führt zu kompakterem und verständlicherem Code.

4.2.2 Vite

Vite (französisch für *schnell*) ist das Build-Tool der Wahl. Es löst ältere Tools wie Webpack ab, indem es während der Entwicklung native ES-Module im Browser nutzt. Das bedeutet, dass der Browser die Importe selbst auflöst, anstatt dass der gesamte Code vor jedem Start gebündelt werden muss. Dies führt zu extrem schnellen Server-Startzeiten, selbst bei wachsender Projektgröße. Für die Produktion nutzt Vite im Hintergrund Rollup, um hochoptimierte, minifizierte Bundles zu erstellen.

4.2.3 React Router (v7)

Da eine SPA physisch nur aus einer Seite besteht, muss die Navigation simuliert werden. **React Router** überwacht die URL im Browser und rendert basierend auf definierten Regeln die entsprechenden Komponenten. Dies ermöglicht Deep-

Linking (Lesezeichen auf Unterseiten) und die Verwendung der Browser-Historie (Zurück-Button), obwohl die Seite nie neu geladen wird.

4.3 Projektsetup und Konfiguration

Das Projekt wurde mit dem Befehl `npm create vite@latest` initialisiert. Die Konfiguration befindet sich in der Datei `vite.config.js`.

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vite.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7   server: {
8     host: true,
9     port: 5173,
10    proxy: {
11      '/api': {
12        target: process.env.VITE_API_TARGET || 'http://localhost:5015',
13        changeOrigin: true
14      }
15    }
16  }
17 })
```

Listing 4.1: Auszug aus der Vite-Konfiguration

Ein kritischer Aspekt dieser Konfiguration ist der **Proxy**: Um während der Entwicklung Cross-Origin Resource Sharing (CORS) Probleme zu vermeiden, leitet der Vite-Entwicklungsserver alle Anfragen, die mit `/api` beginnen, an das Backend (Port 5015) weiter. Die Option `changeOrigin: true` manipuliert dabei den Host-Header der Anfrage so, als käme sie direkt vom Backend-Server selbst.

Die Projektstruktur im Order `src` folgt einer funktionalen Aufteilung:

- `components/`: Beinhaltet UI-Komponenten (z.B. `Chat.jsx`, `Quiz.jsx`). Jede Komponente hat idealerweise ihre eigene CSS-Datei (z.B. `Chat.css`) für isoliertes Styling.
- `services/`: Kapselt die gesamte API-Kommunikation.
- `assets/`: Für statische Dateien.

4.4 Softwarearchitektur und Datenfluss

4.4.1 Unidirektionaler Datenfluss

React erzwingt einen unidirektionalen Datenfluss („One-Way Binding“). Daten fließen immer von Eltern- zu Kind-Komponenten mittels sogenannter „Props“. Um Daten von einem Kind zurück zum Elternteil zu senden (z.B. ein Formular-Submit im `QuizModal`, der die Liste im `Quiz` aktualisiert), übergibt die Elternkomponente eine Callback-Funktion an das Kind.

4.4.2 Routing-Strategie

Das Routing in `App.jsx` definiert die Skelettstruktur der Anwendung. Es wird zwischen öffentlichen Routen (Login, Register) und privaten, geschützten Routen unterschieden.

```

1 <Routes>
2   { /* Öffentliche Routen */ }
3   <Route path="/login" element={<Login setLoggedInUser={setLoggedInUser} />} />
4
5   { /* Geschuetzte Routen - erfordern Login */ }
6   <Route path="/*" element={
7     loggedInUser ? (
8       <Routes>
9         <Route element={<MainLayout handleLogout={handleLogout} user={
10           loggedInUser} />} />
11         <Route index element={<MainDashboard />} />
12         <Route path="files" element={<Files />} />
13         <Route path="chat" element={<Chat />} />
14         { /* ... weitere Routen ... */ }
15       </Routes>
16     ) : (
17       <Navigate to="/login" />
18     )
19   } />
20 </Routes>

```

Listing 4.2: Routing-Struktur in `App.jsx`

Die Verwendung von verschachtelten Routen (`<Route element=<MainLayout ...>>`) erlaubt es, das Grundgerüst (Header, Navigation) einmalig zu definieren. Die Kind-Komponenten werden dann an der Stelle des `<Outlet />` Platzhalters in `MainLayout.jsx` gerendert.

4.5 Datenschicht und API-Integration

Die gesamte Kommunikation mit dem Backend wird über das Modul `src/services/api.js` abgewickelt. Dieses Muster, oft als "SService Layer Pattern" bezeichnet, hat den Vorteil, dass Komponenten keine Details über URLs oder HTTP-Header kennen müssen.

4.5.1 API-Wrapper

Das `api`-Objekt fungiert als Wrapper um die native `fetch`-API. Es implementiert globale Verhaltensweisen:

1. **Basis-URL:** Alle Anfragen sind relativ zu einer zentral konfigurierten Basis-URL.
2. **Credentials:** Durch `credentials: 'include'` sendet der Browser automatisch Cookies (für die `HttpOnly` Session-Cookies) mit jeder Anfrage mit.
3. **Content-Type Handling:** JSON-Payloads werden automatisch serialisiert und der korrekte Header gesetzt.
4. **Zentrale Fehlerbehandlung:** Antworten mit einem HTTP-Statuscode außerhalb des 200er-Bereichs lösen automatisch eine Exception aus. Dies vereinfacht die Fehlerbehandlung in den Komponenten, da `try/catch`-Blöcke verwendet werden können.

```
1 get: async (endpoint, options = {}) => {
2   const response = await fetch(`${API_BASE_URL}${endpoint}`, {
3     credentials: 'include',
4     ...options,
5   });
6   if (!response.ok) {
7     // Fehlerbehandlung: Logging und Werfen eines Errors
8     throw new Error('API Error: ${response.status}');
9   }
10  return await response.json();
11 },
```

Listing 4.3: Generische GET-Methode im API-Service

Zusätzlich stellt der Service spezifische Methoden bereit, die die Geschäftslogik abbilden, beispielsweise `createChatSession`, `updateQuiz` oder `uploadDocument`. Letzteres nutzt `FormData`, um Datei-Uploads (multipart/form-data) zu ermöglichen.

4.6 Implementierung der Kernkomponenten

4.6.1 Authentifizierungssystem

Das Login-System (`Login.jsx`) ist der erste Berührungspunkt für den Nutzer. Das Formular sendet E-Mail und Passwort an den Endpunkt `/Users/login`. Bei Erfolg antwortet das Server-Backend mit einem Session-Cookie (`HttpOnly`) und den Benutzerdaten.

Das Besondere an der Implementierung ist das sofortige Abrufen der Benutzerrollen nach dem Login. Da die Rollen („Admin“, „User“) bestimmen, welche Menüpunkte sichtbar sind, führt das Frontend einen sekundären Aufruf an `/Users/{id}/roles` durch. Das kombinierte Benutzerobjekt wird anschließend im `localStorage` gespeichert. Dies dient nicht der Sicherheit (da der `localStorage` manipuliert werden kann), sondern lediglich der Persistenz des UI-Zustands (z. B. „Ist der Nutzer eingeloggt?“), bis die echte Session beim Neuladen durch `/Users/me` validiert wird.

4.6.2 Dateimanagement (`Files.jsx`)

Die Komponente `Files.jsx` realisiert ein Dokumentenmanagementsystem.
Funktionsweise:

- Beim Laden der Komponente ruft `useEffect` die Liste aller Dokumente ab.
- Ein Suchfeld filtert die lokale Liste in Echtzeit (`filteredFiles.filter(...)`).
- Das Kontextmenü (Drei-Punkte-Menü) wird über den Zustand `activeMenuId` gesteuert, sodass immer nur ein Menü gleichzeitig offen sein kann.

Der Upload-Prozess wird über ein modales Fenster (`UploadDialog.jsx`) realisiert. Hier wählt der Nutzer eine Datei aus, welche zusammen mit der User-ID an den Server gesendet wird.

4.6.3 Interaktives Lernen: Das Quiz-Modul

Das Quiz-System ist eine der komplexesten Komponenten des Frontends. Es besteht aus der Übersichtsliste (`Quiz.jsx`) und dem Editor/Player (`QuizModal.jsx`, `Quiz.jsx` Logik).

4.6.3.1 Datenstruktur

Ein Quiz besteht aus einem Titel und einer Liste von Fragen. Jede Frage enthält wiederum eine Liste von Antworten, wobei eine oder mehrere als korrekt markiert sind. Diese hierarchische Struktur wird im State als verschachteltes Array objekt abgebildet.

4.6.3.2 Ablauf eines Quiz

Der Zustand während eines laufenden Quiz wird durch mehrere State-Variablen gesteuert:

- `currentQuestionIndex`: Index der aktuell angezeigten Frage.
- `score`: Anzahl der bisher korrekt beantworteten Fragen.
- `selectedAnswerId`: Um eine Mehrfachauswahl zu verhindern und visuelles Feedback zu geben.
- `showScore`: Ein Boolean-Flag, das nach der letzten Frage auf `true` gesetzt wird, um die Ergebnisanzeige zu rendern.

Das visuelle Feedback (Grün für richtig, Rot für falsch) wird durch CSS-Klassen gesteuert, die dynamisch basierend auf der Korrektheit der Antwort zugewiesen werden.

4.6.4 Flashcards (Lernkarten)

Die `Flashcards.jsx` Komponente nutzt CSS3 3D-Transformationen, um einen realistischen Umdreh-Effekt zu erzielen.

```
1 .flashcard-scene {  
2   perspective: 1000px; /* Gibt dem 3D-Raum Tiefe */  
3 }  
4 .flashcard-container {  
5   transition: transform 0.6s;  
6   transform-style: preserve-3d;
```

```

7 }
8 .flashcard-scene.flipped .flashcard-container {
9   transform: rotateY(180deg);
10 }

```

Listing 4.4: CSS für den Flip-Effekt

Der Status, welche Karte gerade umgedreht ist, wird in einem `Set` (`flippedCards`) gespeichert. Dies ermöglicht es, mehrere Karten gleichzeitig umzudrehen, im Gegensatz zu einer simplen ID-Variable.

4.6.5 Chat und KI-Integration

Der Chat (`Chat.jsx`) bietet eine Schnittstelle zu einem LLM (Large Language Model). Wichtige Implementierungsdetails:

- **Streaming vs. Polling:** Die aktuelle Implementierung wartet auf die vollständige Antwort des Servers (Request/Response Pattern). Während des Wartens wird ein "Thinking Indicator" (animierte Punkte) eingeblendet (`isLoading` State).
- **Markdown:** Da KI-Modelle oft strukturierten Text (Codeblöcke, Listen) ausgeben, wird `react-markdown` in Kombination mit `rehype-highlight` verwendet. Dies konvertiert die Markdown-Antwort sicher in HTML und bietet Syntax-Highlighting für Code.
- **Multimodaler Input:** Benutzer können Bilder hochladen. Diese werden lokal mittels `URL.createObjectURL()` sofort in der Vorschau angezeigt, bevor sie als Blob an den Server gesendet werden.

4.6.6 Admin Panel und RBAC

Das `AdminPanel.jsx` ist das Kontrollzentrum für Systemadministratoren. Der Zugriff auf diese Route wird auf zwei Ebenen geschützt: 1. **Client-Side Routing:** In `App.jsx` wird geprüft, ob das Benutzerobjekt die Rolle Admin enthält. Falls nicht, wird der Nutzer sofort umgeleitet. 2. **Server-Side API:** Selbst wenn ein Nutzer die UI umgehen würde, lehnt das Backend Anfragen an admin-spezifische Endpunkte ab.

Das Panel ermöglicht CRUD-Operationen (Create, Read, Update, Delete) für Benutzer. Besonders hervorzuheben ist die verschachtelte Datenansicht: Klickt ein Admin auf einen Benutzer, werden dessen Dateien asynchron nachgeladen

(`fetchUserFiles`) und in einer Detailansicht dargestellt. Dies vermeidet das Laden unnötiger Daten für die Hauptübersichtstabelle.

4.7 Benutzeroberfläche und Design (UI/UX)

4.7.1 Styling-Strategie

Das Projekt verwendet natives CSS mit CSS-Variablen (Custom Properties) für das Theming. Dies bietet maximale Flexibilität ohne den Overhead großer Frameworks. Globale Styles sind in `index.css` definiert, während komponentenspezifische Styles (z. B. `Chat.css`) lokal importiert werden.

4.7.2 Dark Mode Implementierung

Der Dunkelmodus ist nicht nur eine kosmetische Spielerei, sondern eine zentrale Anforderung. Er wurde über CSS-Variablen realisiert, die je nach Attribut am HTML-Tag ihre Werte ändern.

```
1 :root {
2   /* Standard (Dark Mode) */
3   --primary-color: #222831;
4   --text-color: #FFFFFF;
5 }
6
7 [data-theme='light'] {
8   /* Light Mode Overrides */
9   --primary-color: #FFFFFF;
10  --text-color: #222831;
11 }
12
13 body {
14   background-color: var(--primary-color);
15   color: var(--text-color);
16 }
```

Listing 4.5: Theme-Switching mittels CSS-Variablen

Ein `useEffect`-Hook in der `App.jsx` prüft beim Start den `localStorage`, ob der Nutzer eine Präferenz gespeichert hat, und setzt das Attribut entsprechend. Dies verhindert den Flash of Incorrect Theme"(FOUC).

4.8 Detaillierte Analyse ausgewählter Module

Um die technische Tiefe der Implementierung zu verdeutlichen, werden im Folgenden drei Kernmodule detailliert anhand ihres Quellcodes analysiert.

4.8.1 Quiz-Editor Logik (QuizModal.jsx)

Der Quiz-Editor ermöglicht das Erstellen und Bearbeiten von verschachtelten Datenstrukturen (Quiz -> Fragen -> Antworten). Die Herausforderung hierbei ist das State-Management einer mehrdimensionalen Struktur.

4.8.1.1 Zustandsverwaltung

Der State `questions` ist ein Array von Objekten. Jedes Objekt repräsentiert eine Frage und enthält wiederum ein Array von `answers`.

```

1 const [questions, setQuestions] = useState([
2   {
3     text: '',
4     answers: [{ text: '', isCorrect: false }]
5   }
6 ]);

```

Listing 4.6: Initialer State im QuizModal

4.8.1.2 Dynamische Formular-Manipulation

Beim Ändern eines Antworttextes darf der State nicht direkt mutiert werden. Stattdessen muss eine tiefe Kopie (Deep Copy) oder zumindest eine flache Kopie der betroffenen Ebenen erstellt werden. Die Funktion `handleAnswerChange` demonstriert das Prinzip der Immutability in React:

```

1 const handleAnswerChange = (questionIndex, answerIndex, event) => {
2   // 1. Kopie des Fragen-Arrays
3   const newQuestions = [...questions];
4   // 2. Direkter Zugriff auf die Antwort (Referenz bleibt erhalten, aber
5     // Inhalt wird ueberschrieben)
6   // Hinweis: Fuer saubere Immutability muesste auch answers kopiert
7     // werden.
8   // In React funktioniert dies jedoch oft, solange das Top-Level-Array
9     // neu ist.
10  newQuestions[questionIndex].answers[answerIndex].text = event.target.
11    value;

```

```
8 // 3. Setzen des neuen States loest Re-Render aus
9 setQuestions(newQuestions);
10 };
```

Listing 4.7: Update-Logik für verschachtelte Antworten

4.8.1.3 Validierung und Speichern

Beim Speichern (`handleSubmit`) werden die Daten an die API gesendet. Hierbei wird unterschieden, ob ein neues Quiz erstellt (POST) oder ein bestehendes bearbeitet (PUT) wird.

4.8.2 Flashcards: 3D-Interaktion und State

Die Komponente `Flashcards.jsx` verwendet eine interessante Datenstruktur für den Zustand der umgedrehten Karten. Anstatt für jede Karte einen Boolean im Objekt zu speichern (was eine Änderung des Datenmodells erfordern würde), wird ein `Set` verwendet, das die IDs der umgedrehten Karten speichert.

```
1 const [flippedCards, setFlippedCards] = useState(new Set());
2
3 const flipCard = (id) => {
4   setFlippedCards(prevFlippedCards => {
5     const newFlippedCards = new Set(prevFlippedCards);
6     if (newFlippedCards.has(id)) {
7       newFlippedCards.delete(id); // Karte umdrehen (zurueck)
8     } else {
9       newFlippedCards.add(id); // Karte aufdecken
10    }
11    return newFlippedCards;
12  });
13 };
```

Listing 4.8: Nutzung eines Sets fuer UI-State

Diese Methode ist performant und trennt den UI-Zustand (ist die Karte gedreht?) sauber von den eigentlichen Geschäftsdaten (Frage/Antwort). Beim Rendern wird `flippedCards.has(card.id)` geprüft, um die CSS-Klasse `.flipped` bedingt hinzuzufügen.

4.8.3 Admin Panel: Kaskadierende Datenabfragen

Im `AdminPanel.jsx` muss beim Klick auf einen Benutzer dessen Dateiliste geladen werden. Dies ist ein klassisches Master-DetailSzenario. Um unnötigen

Netzwerktraffic zu vermeiden, werden die Dateien nicht initial für alle Benutzer geladen (Lazy Loading").

```

1  const handleClick = (user) => {
2      setSelectedUser(user);
3      setError('');
4      setEditMode(false);
5      // Erst beim Klick werden die Dateien nachgeladen
6      fetchUserFiles(user.userId);
7  };
8
9  const fetchUserFiles = async (userId) => {
10     try {
11         setLoading(true); // Ladeindikator an
12         const files = await api.get(`/Users/${userId}/documents`);
13         setUserFiles(files);
14     } catch (err) {
15         setError('Could not load user files.');
```

Listing 4.9: Asynchrones Laden von Detaildaten

Diese Implementierung sorgt für eine schnelle initiale Ladezeit der Benutzertabelle, da die (potenziell große) Menge an Dateimetadaten erst bei Bedarf (Ön-Demand") abgerufen wird.

4.9 Deployment und Betrieb

Für den produktiven Einsatz wird die Frontend-Anwendung containerisiert. Dies gewährleistet eine konsistente Laufzeitumgebung unabhängig vom Host-System.

4.9.1 Docker-Containerisierung

Es wird ein „Multi-Stage Build“ Prozess verwendet, um das finale Image so klein wie möglich zu halten.

1. **Build-Stage:** Ein Node.js Image wird verwendet, um die Abhängigkeiten zu installieren und den Build-Prozess (`npm run build`) auszuführen. Das Ergebnis ist ein Ordner `dist`, der nur die statischen Dateien (HTML, CSS, JS) enthält.

2. **Production-Stage:** Ein leichtgewichtiger Nginx-Webserver wird als Basis genommen. Die Dateien aus der Build-Stage werden in das Verzeichnis `/usr/share/nginx/html` kopiert.

```
1 # Stage 1: Build
2 FROM node:20-alpine as build
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npm run build
8
9 # Stage 2: Serve
10 FROM nginx:alpine
11 COPY --from=build /app/dist /usr/share/nginx/html
12 COPY nginx.conf /etc/nginx/conf.d/default.conf
13 EXPOSE 80
14 CMD ["nginx", "-g", "daemon off;"]
```

Listing 4.10: Dockerfile für das Frontend

4.9.2 Nginx Konfiguration

Da es sich um eine SPA handelt, muss der Webserver so konfiguriert werden, dass er für alle Routen (z.B. `/files`, `/chat`) immer die `index.html` ausliefert (Fallback"). Andernfalls würde ein direkter Aufruf einer Unterseite zu einem 404-Fehler führen, da diese Dateien physisch nicht existieren.

4.10 Herausforderungen und Lösungen

Während der Entwicklung traten verschiedene technische Herausforderungen auf, die spezifische Lösungen erforderten.

4.10.1 Synchronisation des UI-Status

Problem: Wenn ein Nutzer ein Quiz bearbeitet, muss die Liste der Quizzes im Hintergrund aktuell bleiben. **Lösung:** Callback-Funktionen (z.B. `onQuizSaved`) werden an die Modal-Komponenten übergeben. Nach erfolgreicher API-Antwort ruft das Modal diese Funktion auf, und die Elternkomponente aktualisiert ihren lokalen State, ohne einen erneuten Fetch auszuführen. Optimistic UI Updates wurden in Betracht gezogen, aber zugunsten der Datenkonsistenz verworfen.

4.10.2 Performance bei großen Dateilisten

Problem: Das Rendern von hunderten Datei-Icons führte zu Verzögerungen.

Lösung: Durch die Pagination im Backend (noch zu implementieren) und Virtualisierung der Listen im Frontend (Rendering nur der sichtbaren Elemente) lässt sich dies optimieren. Aktuell wurde die Performance durch das Entfernen unnötiger Re-Renders mittels `React.memo` und `useCallback` verbessert.

4.11 Ausblick und Erweiterungsmöglichkeiten

Das aktuelle Frontend bietet eine solide Basis, kann jedoch in Zukunft erweitert werden:

- **Automatisierte Tests:** Integration von `Vitest` für Unit-Tests der Logik und `Cypress` für End-to-End Tests der Benutzeroberfläche.
- **PWA (Progressive Web App):** Durch Hinzufügen eines Service Workers und eines Web App Manifests könnte die Anwendung offline-fähig gemacht und auf Mobilgeräten installiert werden.
- **Internationalisierung (i18n):** Nutzung von Bibliotheken wie `react-i18next`, um die Oberfläche in mehreren Sprachen anzubieten.

4.12 Fazit

Die Frontend-Implementierung von EduShelf demonstriert die Leistungsfähigkeit moderner Webtechnologien. Durch den Einsatz von React und einer komponentenbasierte Architektur entstand eine wartbare, skalierbare und benutzerfreundliche Anwendung. Die Integration komplexer Features wie Real-Time-Editoren, KI-Chat und interaktive Lernmodule in einer Single Page Application zeigt, dass Webanwendungen nativen Programmen in nichts nachstehen müssen.

Zeiterfassung



Abbildung 4.1: AI und iOS Zeitaufwand

Gesamt: 263,17h

Freizeit: 228,48h

5 Backend Implementierung

Die technische Umsetzung des EduShelf-Backends stellt das fundamentale Gerüst der Anwendung dar. Es wurde als monolithische ASP.NET Core Web API konzipiert, die als zentrale Schnittstelle zwischen dem Frontend, der Datenbank, dem KI-Modell und dem Dateispeicherdienst dient. Diese Entscheidung für einen monolithischen Ansatz wurde bewusst getroffen, um die Entwicklungskomplexität initial gering zu halten (*Keep It Simple, Stupid - KISS*), während durch den Einsatz moderner .NET 8 Technologien gleichzeitig eine hohe Performance, Skalierbarkeit und Wartbarkeit gewährleistet wird.

In den folgenden Abschnitten wird die Architektur, der Technologie-Stack, das Datenmodell sowie die spezifischen Implementierungsdetails der Kernkomponenten detailliert erläutert.

5.1 Systemarchitektur

Die Architektur des Systems folgt dem bewährten Muster einer *Layered Architecture* (Schichtenarchitektur), um eine saubere Trennung der Verantwortlichkeiten (*Separation of Concerns*) zu erzielen. Dies erleichtert nicht nur die Testbarkeit mittels Unit- und Integrationstests, sondern ermöglicht auch zukünftige Erweiterungen oder eine potenzielle Migration zu einer Microservices-Architektur, ohne das Gesamtsystem neu schreiben zu müssen.

Die Anwendung gliedert sich in vier Hauptschichten:

5.1.1 Presentation Layer (Controllers)

Diese oberste Schicht nimmt HTTP-Anfragen (Requests) entgegen und gibt HTTP-Antworten (Responses) zurück. Sie fungiert als Tor zur Außenwelt.

- **Verantwortlichkeit:** Routing, Deserialisierung von JSON-Body-Daten, Validierung von Eingaben und Formatierung der Ausgaben.

- **Validierung:** Mithilfe der Bibliothek *FluentValidation* werden eingehende Datenmodelle (DTOs) noch vor der Verarbeitung auf Korrektheit geprüft (z.B. `UserRegisterValidator`, der sicherstellt, dass E-Mail-Adressen gültig sind und Passwörter Mindestanforderungen erfüllen).
- **Controller:** Die Controller sind schlank gehalten (*Thin Controllers*) und delegieren die eigentliche Geschäftslogik sofort an die Service-Schicht. Beispiel: Der `ChatController` leitet die Benutzeranfrage direkt an den `ChatService` weiter.

5.1.2 Business Logic Layer (Services)

Hier residiert das "Gehirn" der Anwendung. Alle fachlichen Regeln und Prozesse sind in dieser Schicht implementiert.

- **Service-Klassen:** Jede Domäne hat ihren eigenen Service (z.B. `AuthService`, `IndexingService`, `RAGService`).
- **Orchestrierung:** Services koordinieren den Datenfluss zwischen der Datenbank, externen Diensten (wie MinIO oder Ollama) und der Präsentationsschicht.
- **Dependency Injection:** Alle Services werden über das in .NET integrierte Dependency Injection (DI) Container registriert, was die Austauschbarkeit von Implementierungen zur Laufzeit ermöglicht.
- **Hintergrundverarbeitung:** Für langlaufende Aufgaben wie das Indizieren von Dokumenten wird eine `BackgroundJobQueue` eingesetzt. Dies entkoppelt rechenintensive Prozesse vom HTTP-Request-Zyklus und verhindert Timeouts im Frontend. Der `DocumentService` reiht Aufgaben ein, die dann von einem `BackgroundService` asynchron abgearbeitet werden.

5.1.3 Data Access Layer (Data)

Diese Schicht abstrahiert den Zugriff auf die persistente Datenspeicherung.

- **Entity Framework Core (EF Core):** Als Object-Relational Mapper (ORM) ermöglicht EF Core die Definition des Datenmodells in C#-Klassen. SQL-Queries werden zur Laufzeit generiert (LINQ to SQL), was die Entwicklung beschleunigt und vor SQL-Injection schützt.

- **Repositories:** In diesem Projekt wird das `DbContext`-Pattern direkt verwendet, was bei EF Core oft äquivalent zum Repository-Pattern bzw. Unit-of-Work-Pattern ist.

5.1.4 Infrastructure Layer

Diese Schicht beinhaltet die technische Anbindung an externe Systeme und Infrastrukturkomponenten.

- **MinIO:** Für die Speicherung unstrukturierter Daten (Blobs) wie PDF-Dokumente oder Bilder.
- **Ollama:** Schnittstelle zum lokalen KI-Modell.
- **E-Mail-Dienst:** Implementierung des `SmtpEmailService` zum Versenden von Bestätigungs-E-Mails.

5.2 Technologie-Stack

Das Backend setzt auf einen modernen, vollständig Open-Source-basierten Stack, der speziell für performante, cloud-native und KI-gestützte Anwendungen optimiert wurde.

- **ASP.NET Core 8:** Das zugrundeliegende Framework bietet native Unterstützung für asynchrone Programmierung (`async/await`), was für I/O-lastige Operationen (Datenbank, Netzwerk) essenziell ist. Es ist plattformunabhängig und läuft in Linux-Containern.
- **PostgreSQL + pgvector:** PostgreSQL wurde aufgrund seiner Robustheit und Erweiterbarkeit gewählt. Die entscheidende Komponente für die RAG-Funktionalität ist die Erweiterung `pgvector`. Sie ermöglicht das Speichern von hochdimensionalen Vektoren (Embeddings) direkt in der Datenbank und führt Ähnlichkeitssuchen (Nearest Neighbor Search) mittels Kosinus-Distanz oder Euklidischer Distanz performant durch.
- **Ollama:** Ein lokaler Server zur Ausführung von Large Language Models (LLMs) wie Llama 3 oder Phi-3. Durch die lokale Ausführung bleiben sensible Dokumentendaten auf dem Server und werden nicht an Drittanbieter-APIs (wie OpenAI) gesendet, was den Datenschutz massiv verbessert.

- **MinIO:** Ein S3-kompatibler Object Storage. Hochgeladene Dateien werden nicht als BLOB in der relationalen Datenbank gespeichert (was diese aufblähen würde), sondern effizient im Dateisystem über MinIO verwaltet.
- **Microsoft Semantic Kernel:** Ein SDK, das als Abstraktionsschicht für KI-Dienste dient. Es vereinfacht die Integration verschiedener KI-Modelle und bietet Mechanismen für Prompt-Templating, Memory-Management (Vektorspeicher-Abstraktion) und Function Calling.
- **PdfPig & Docnet.Core:** Spezialisierte Bibliotheken zur Extraktion von Text aus PDF-Dateien. Docnet.Core ist ein Wrapper um die native 'pdfium'-Bibliothek und bietet extrem hohe Performance beim Rendern, während PdfPig reines C# ist und gut für Text-Extraktion geeignet ist.

5.3 Projekt-Setup und Konfiguration

Analog zum Frontend erfolgt der Einstieg in die Anwendung über eine definierte Startup-Logik, die in der `Program.cs` und den Konfigurationsdateien verankert ist.

5.3.1 Programm-Initialisierung (`Program.cs`)

Die `Program.cs` ist der "Entry Point" der ASP.NET Core Anwendung. Hier wird der "Generic Host" gebaut, der Dependency Injection Container befüllt und die HTTP-Request-Pipeline (Middleware) konfiguriert.

```

1      var builder = WebApplication.CreateBuilder(args);
2
3      // 1. Service Registrierung (DI)
4      builder.Services.AddApplicationServices(builder.Configuration);
5      builder.Services.AddControllers();
6      builder.Services.AddEndpointsApiExplorer();
7      builder.Services.AddSwaggerGen();
8
9      var app = builder.Build();
10
11     // 2. HTTP Request Pipeline
12     if (app.Environment.IsDevelopment())
13     {
14         app.UseSwagger();
15         app.UseSwaggerUI();
16     }
17
18     app.UseHttpsRedirection();
19     app.UseCors("AllowNextJs");

```

```
20 app.UseAuthentication();
21 app.UseAuthorization();
22 app.MapControllers();
23
24 app.Run();
```

Listing 5.1: Auszug aus der Middleware-Konfiguration

5.3.2 Konfiguration (appsettings.json)

Umweltspezifische Variablen werden aus der `appsettings.json` oder Umgebungsvariablen geladen. Dies folgt dem "12-Factor AppPrinzip. Wichtige Sektionen sind:

- **ConnectionStrings:** Verbindung zur PostgreSQL Datenbank.
- **AIService:** API-Keys und Modell-Namen für Ollama.
- **SmtpConfig:** Zugangsdaten für den E-Mail-Versand.
- **JwtConfig:** (Legacy) Secret Keys für JWTs, falls verwendet.

5.4 Datenmodelle und Datenbankdesign

Das Datenmodell wurde entworfen, um sowohl relationale Benutzerdaten als auch unstrukturierte Dokumenteninhalte effizient zu verwalten. EF Core Migrationen (**Migrations** Ordner) stellen sicher, dass das Datenbankschema synchron zum Code bleibt.

5.4.1 Kernelemente

- **User:** Die zentrale Identitätseinheit. Enthält `Username`, `Email`, `PasswordHash` (gespeichert als BCrypt-Hash), sowie Felder für die E-Mail-Verifizierung (`IsEmailConfirmed`, `EmailConfirmationToken`).
- **Role & UserRole:** Implementierung einer Many-to-Many Beziehung für ein rollenbasiertes Zugriffssystem (RBAC). Standardrollen sind z.B. `SStudent` oder `Admin`.

- **Document:** Repräsentiert Metadaten eines hochgeladenen Dokuments. Das Feld `Path` verweist auf den Speicherort im MinIO-Bucket. Es besteht eine 1:n-Beziehung zum User.
- **DocumentShare:** Ermöglicht das Teilen von Dokumenten zwischen Benutzern (Many-to-Many via Join-Table). Diese Entität steuert den Zugriff für Nicht-Besitzer.
- **Cascading Deletes:** Die Datenbankkonfiguration (`OnModelCreating`) stellt sicher, dass beim Löschen eines Nutzers alle abhängigen Daten (Dokumente, Chats, Favoriten) automatisch entfernt werden (`DeleteBehavior.Cascade`), um Datenwaisen zu vermeiden.
- **DocumentChunk:** Dies ist die wichtigste Entität für die semantische Suche. Ein Dokument wird in viele kleine Abschnitte zerlegt.
 - **Content:** Der rohe Textinhalt des Abschnitts.
 - **Embedding:** Ein Vektor vom Typ `vector(768)`, der die semantische Bedeutung des Textes repräsentiert.
 - **Page:** Referenz zur ursprünglichen Seite im Dokument (für Zitate).
- **ChatSession & ChatMessage:** Bilden den Verlauf einer Konversation ab. `ChatSession` gruppiert Nachrichten, während `ChatMessage` den eigentlichen Inhalt (User-Input oder AI-Response) speichert.

5.5 API Design und Controller

Die Kommunikation mit dem Frontend erfolgt über eine RESTful API. Die Controller sind thematisch gruppiert und folgen REST-Konventionen.

5.5.1 Wichtige Endpunkte

- **DocumentsController:** Verwaltet den Lebenszyklus von Dokumenten. Wichtige Aktionen sind Upload (POST), Download (GET stream), und Sharing (POST `/share`).
- **Indexing-Trigger:** Nach dem erfolgreichen Upload stößt der `DocumentsController` sofort einen Hintergrundjob an, der das Indexieren startet.

- **ChatController:** Handhabt Chat-Sitzungen. Ein POST auf `/message` löst den gesamten RAG-Workflow aus.
- **AuthController:** Zuständig für Login, Registrierung (mit E-Mail-Bestätigungsflow) und Token-Verwaltung.

5.6 Sicherheitskonzepte

Sicherheit war ein integraler Bestandteil des Designs (SSecurity by Design").

5.6.1 Authentifizierung & Autorisierung

Die Authentifizierung erfolgt über Cookies (`CookieAuthenticationDefaults`).

- **Login-Prozess:** Der Nutzer sendet Anmeldedaten an `/api/auth/login`. Der `AuthService` überprüft den Passwort-Hash mittels BCrypt. Bei Erfolg wird ein verschlüsseltes HTTP-only Cookie gesetzt, das den Session-Cookie darstellt.
- **Claims-Based Identity:** Das Cookie enthält Claims" (Aussagen) über den Benutzer, wie z.B. Benutzer-ID, Name und Rollen.
- **Middleware:** Die Standard-ASP.NET Core Authentifizierungsmiddleware entschlüsselt das Cookie bei jedem Request und rekonstruiert das `User`-Objekt (`HttpContext.User`).

5.6.2 Datenvalidierung

Eingehende Daten werden rigoros validiert. Die Bibliothek *FluentValidation* erlaubt die Definition von komplexen Regeln. Beispiel `UserRegisterValidator`:

- Username: Muss 3-20 Zeichen lang sein.
- Email: Muss ein gültiges E-Mail-Format haben.
- Password: Mindestens 6 Zeichen.

Diese Validierung geschieht automatisch vor dem Controller-Aufruf. Ungültige Anfragen werden sofort mit HTTP 400 (Bad Request) abgelehnt.

5.6.3 Error Handling

Ein globales `ErrorHandlingMiddleware` fängt alle unbehandelten Ausnahmen ab. Dies verhindert, dass interne Implementierungsdetails (Stack Traces) an den Client durchsickern. Stattdessen werden standardisierte JSON-Fehlermeldungen und passende HTTP-Statuscodes zurückgegeben (z.B. `NotFoundException` → 404, `UnauthorizedAccessException` → 401).

5.7 Ingestion-Pipeline (Datenverarbeitung)

Damit Dokumente durchsuchbar werden, müssen sie eine komplexe Verarbeitungspipeline durchlaufen. Dieser Prozess wird vom `IndexingService` gesteuert und ist rechenintensiv.

5.7.1 1. Textextraktion (OCR & Parsing)

Der `FileParsingService` analysiert den Dateityp und wählt die passende Strategie:

- **PDF-Verarbeitung mit PdfPig:** Für PDF-Dateien wird die Bibliothek *UglyToad.PdfPig* verwendet. Sie erlaubt den Zugriff auf den rohen Textstream sowie die Extraktion von Bildern.
- **Hybride Bildanalyse:** Eingebettete Bilder werden isoliert und an ein lokales Vision-Model (via Ollama) gesendet. Der Prompt "Describe this image in detail" generiert eine textuelle Beschreibung, die in den Dokumentenindex einfließt. Dies ermöglicht die semantische Suche nach Bildinhalten (z.B. Diagramme, Graphen).
- **DOCX:** Nutzung des Open XML SDKs zum strukturierten Auslesen des `MainDocumentPart`.

5.7.2 2. Chunking (Intelligente Segmentierung)

LLMs haben ein begrenztes Kontextfenster, und die semantische Suche verliert bei zu großen Textblöcken an Präzision. Der `TextChunker` implementiert daher eine mehrstufige Strategie:

1. **Satz-Tokenisierung:** Der Text wird zunächst anhand von Satzzeichen (. ! ?) in einzelne Sätze zerlegt.
2. **Rekombination:** Diese Sätze werden iterativ zu einem Chunk zusammengefügt, bis eine konfigurierte Grenze von ca. 1024 Zeichen erreicht ist. Dies stellt sicher, dass semantische Zusammenhänge (Sätze) nicht willkürlich in der Mitte durchschnitten werden.
3. **Token-Limit:** Zusätzlich wird sichergestellt, dass die Chunks eine maximale Token-Anzahl (für das Embedding-Model) nicht überschreiten.

5.7.3 3. Embedding und Vektorspeicherung

Jeder Text-Chunk wird in einen hochdimensionalen Vektor umgewandelt und effizient gespeichert.

- **Embedding-Modell:** Nutzung von state-of-the-art Modellen wie *nomi-embed-text*, die semantische Ähnlichkeiten im Vektorraum abbilden.
- **HNSW Index:** In der PostgreSQL-Datenbank wird für die Spalte `Embedding` ein HNSW (Hierarchical Navigable Small World) Index angelegt.

```
modelBuilder.Entity<DocumentChunk>()  
    .HasIndex(dc => dc.Embedding)  
    .HasMethod("hnsw")  
    .HasOperators("vector_12_ops");
```

Dieser Index ermöglicht eine extrem schnelle Nearest-Neighbor-Suche (ANN) auch bei Millionen von Vektoren, da nicht die gesamte Tabelle gescannt werden muss.

5.8 Detaillierte Analyse ausgewählter Module

Um die technische Tiefe zu verdeutlichen, wird hier exemplarisch die Implementierung sicherheitskritischer und komplexer Logik analysiert.

5.8.1 Authentifizierung: Secure Cookies und Hashing

Die Sicherheit der Benutzerdaten hat oberste Priorität. Der `AuthService` implementiert daher bewährte Industriestandards.

5.8.1.1 Passwort-Hashing (BCrypt)

Passwörter werden niemals im Klartext gespeichert. Stattdessen wird der BCrypt-Algorithmus verwendet, der durch seinen konfigurierbaren "Work Factor"(Kostenfaktor) resistent gegen Brute-Force- und Rainbow-Table-Angriffe ist.

```
1 // user.PasswordHash kommt aus der DB, passwordDto.Password vom User
2 bool isValid = BCrypt.Net.BCrypt.Verify(passwordDto.Password, user.
    PasswordHash);
3 if (!isValid)
4 {
5     throw new UnauthorizedAccessException("Invalid credentials");
6 }
```

Listing 5.2: Verifizierung im AuthService

5.8.1.2 Cookie-Konstruktion

Nach erfolgreichem Login wird kein JWT im LocalStorage (Frontend) abgelegt (XSS-Gefahr), sondern ein **HttpOnly Cookie** gesetzt.

- **HttpOnly:** Das Cookie kann nicht per JavaScript ausgelesen werden.
- **Secure:** Wird nur über HTTPS übertragen.
- **SameSite=Strict:** Das Cookie wird nur an den eigenen Origin gesendet (Schutz vor CSRF).

5.8.2 Ingestion-Pipeline: HNSW Indexierung

Wie in Abschnitt 6 beschrieben, nutzt das System einen HNSW-Index. Die technische Herausforderung liegt hier in der Konfiguration der Operatorenklasse für `pgvector`. Code-seitig wird dies in der `OnModelCreating`-Methode des `ApiDbContext` gelöst, um sicherzustellen, dass PostgreSQL den Index korrekt für L2-Distanz-Berechnungen optimiert. Dies ist ein direktes Mapping von C#-Code auf datenbankspezifische Features (Code-First Migration").

5.9 RAG (Retrieval-Augmented Generation) Workflow

Der RAG-Prozess kombiniert Informationsrückgewinnung mit Textgenerierung.

5.9.1 Retrieval-Strategie

Der `RetrievalService` implementiert eine verfeinerte Suchstrategie, die sich der Komplexität der Anfrage anpasst:

- **Intent-Basierte Filterung:** Der `IntentDetectionService` analysiert die Benutzeranfrage vorab. Er ermittelt, ob der Nutzer explizit in einem bestimmten Dokument suchen will. Falls ja, wird ein Datenbank-Filter auf die `DocumentId` gesetzt.
- **Dynamisches K:** Die Anzahl der abzurufenden Chunks (k) ist nicht statisch. Sie basiert auf der Wortanzahl der Query:
 - < 10 Wörter: $k = 5$ (kurze Fragen)
 - $10 - 30$ Wörter: $k = 10$ (mittlere Fragen)
 - > 30 Wörter: $k = 20$ (komplexe Anweisungen)

Dies optimiert die Balance zwischen Kontextmenge und Noise".

- **Ranking:** Die Datenbank berechnet mittels `Embedding <-> QueryEmbedding` den euklidischen Abstand (L2-Distanz) und liefert die relevantesten Treffer.

5.9.2 Prompt Engineering und Kontext-Management

Der `PromptGenerationService` baut den finalen Prompt zusammen und muss dabei strikte Limits einhalten.

- **Token-Limitierung (Tiktoken):** Um das Kontextfenster des LLMs (z.B. 4096 Token) nicht zu sprengen, wird der `Tiktoken`-Tokenizer (kompatibel mit GPT-4/cl100k_base) verwendet. Er zählt exakt die Token der abgerufenen Chunks und schneidet den Kontext präzise ab, bevor er an das Modell gesendet wird.
- **System Prompt:** Eine strikte System-Instruktion ("You are a helpful assistant...") zwingt das Modell dazu, Antworten 'ausschließlich' auf dem bereitgestellten Kontext zu basieren, um Halluzinationen zu minimieren.

5.10 Infrastruktur & Deployment

Das System ist vollständig containerisiert ("Dockerized"), um eine konsistente Umgebung von der Entwicklung bis zur Produktion zu gewährleisten.

5.10.1 Docker-Komposition

Die `docker-compose.yml` definiert das Zusammenspiel der Dienste und deren spezifische Konfiguration für die Produktionsumgebung.

5.10.1.1 Datenbank (PostgreSQL mit pgvector)

Für die persistente Speicherung und Vektorsuche wird ein spezialisiertes Image verwendet:

- **Image:** `pgvector/pgvector:pg16` - Dies ist essentiell, da das Standard-Postgres-Image die `vector`-Erweiterung nicht enthält.
- **Ports:** Das Mapping `5433:5432` vermeidet Konflikte mit lokalen Postgres-Instanzen auf dem Host.

- **Healthcheck:** Ein integrierter Healthcheck (`pg_isready`) stellt sicher, dass abhängige Container (wie die API) erst starten, wenn die Datenbank vollständig hochgefahren ist.
- **Persistenz:** Das Volume `postgres_data` sichert die relationalen Daten und Vektor-Indizes über Container-Neustarts hinweg.

5.10.1.2 KI-Inferenz (Ollama)

Der Ollama-Container ist für die lokale Ausführung der LLMs zuständig und benötigt direkten Hardware-Zugriff für performante Inferenz:

- **GPU-Passthrough:** Über die `deploy.resources`-Sektion wird der Nvidia-Treiber durchgereicht:

```
reservations:
  devices:
    - driver: nvidia
      count: 1
      capabilities: [gpu]
```

Dies ermöglicht die Nutzung von CUDA-Kernen, was die Antwortzeit (Time-to-First-Token) drastisch reduziert (von Sekunden auf Millisekunden).

- **Persistenz:** Das Volume `ollama_data` speichert die heruntergeladenen Modelle (z.B. Llama 3) im `/root/.ollama` Verzeichnis, um erneute Downloads bei Container-Updates zu vermeiden.
- **Keep-Alive:** Die Umgebungsvariable `OLLAMA_KEEP_ALIVE` verhindert, dass das Modell nach kurzer Inaktivität aus dem VRAM entladen wird, was die Latenz bei aufeinanderfolgenden Anfragen minimiert.

5.10.1.3 Object Storage (MinIO)

MinIO dient als skalierbarer, S3-kompatibler Dateispeicher.

- **Ports:**
 - 9000: API-Port für die Kommunikation mit dem Backend (S3-Protokoll).

- 9001: Web-Interface (Console) zur manuellen Verwaltung von Buckets und Dateien.
- **Command:** Der Startbefehl `server /data -console-address :9001` aktiviert das Web-UI explizit.
- **Volumes:** `minio_data` speichert die physischen Dateien (PDFs, Bilder) persistent.

5.10.2 Dockerfile

Das `Dockerfile` der API nutzt Multi-Stage Builds, um das Image klein zu halten:

1. **Build Stage:** Nutzt das vollständige SDK Image (`mcr.microsoft.com/dotnet/sdk:8.0`), um den Code zu kompilieren und Abhängigkeiten (NuGet Pakete) zu laden.
2. **Runtime Stage:** Nutzt das schlanke Runtime Image (`mcr.microsoft.com/dotnet/aspnet:8.0`), welches nur die zum Ausführen nötigen Komponenten enthält. Das kompilierte Artefakt aus Stage 1 wird hierhin kopiert.

Dies reduziert die Image-Größe drastisch und verbessert die Sicherheit, da keine Compiler-Tools im Produktions-Container vorhanden sind.

5.11 Herausforderungen und Lösungen

Während der Backend-Entwicklung mussten mehrere technische Hürden genommen werden.

5.11.1 Blockierende Operationen bei großen Dateien

Problem: Das Indexieren eines 500-seitigen PDF-Skripts dauert mehrere Minuten. Ein synchroner HTTP-Request würde hierbei in ein Timeout laufen ("Blocking I/O"). **Lösung:** Einführung einer `BackgroundJobQueue`. Der

Upload-Endpoint nimmt die Datei entgegen, speichert sie in MinIO und reiht nur eine "Job-ID" in die Queue ein, bevor er sofort `202 Accepted` zurückgibt. Ein Hintergrunddienst ("Worker") arbeitet diese Queue asynchron ab.

5.11.2 LLM Kontext-Fenster-Limits

Problem: GPT-4 und andere Modelle haben ein festes Limit an Tokens (z.B. 4096 oder 8192). Ein blindes Einfügen von Dokumenten würde zu Abstürzen führen. **Lösung:** Implementierung des `TokenService` mit *Tiktoken*. Bevor ein Prompt generiert wird, werden die Tokens der abgerufenen Chunks gezählt. Ist das Limit erreicht, werden die am wenigsten relevanten Chunks verworfen (SSmart Truncation).

5.11.3 Datenkonsistenz bei Löschoperationen

Problem: Wenn ein User gelöscht wird, dürfen dessen Dokumente und Vektoren nicht als "Datenmüll" in der Datenbank verbleiben. **Lösung:** Nutzung von `OnDelete(DeleteBehavior.Cascade)` in EF Core. Da auch die Vektoren (`DocumentChunk`) über Foreign Keys hart an das `Document` und dieses an den `User` gekoppelt ist, löscht ein einziger SQL-Befehl (`DELETE FROM Users WHERE Id=...`) kaskadierend den gesamten assoziierten Datenbaum.

5.12 Qualitätssicherung und Tests

Die Stabilität des Backends wird durch automatisierte Tests gewährleistet.

5.12.1 Unit Tests (xUnit)

Die Geschäftslogik (Services) wird isoliert getestet. Externe Abhängigkeiten wie Datenbank oder File-System werden mittels `Moq` gemockt. Beispiel: Ein Test für den `FlashcardService` prüft, ob die richtigen DTOs zurückgegeben werden, ohne dass dabei eine echte KI angefragt wird (Mocking des `IChatCompletionService`).

5.12.2 Integration Tests

Für Controller und die Datenbankinteraktion werden Integrationstests verwendet. Hierbei wird mittels `WebApplicationFactory` ein echter Test-Server im Speicher hochgefahren, der gegen eine In-Memory-Datenbank oder eine Docker-Test-Instanz läuft. Dies stellt sicher, dass die Middleware-Pipeline, das Routing und das Entity Framework korrekt zusammenspielen.

5.13 Fazit

Das Backend von EduShelf stellt ein robustes, skalierbares Fundament für die Lernplattform dar. Durch die Kombination von bewährten Mustern (Layered Architecture, DI) mit modernsten KI-Technologien (Vektor-Datenbanken, RAG) konnte eine leistungsfähige Lösung für das Dokumentenmanagement und interaktive Lernen geschaffen werden.

Zeiterfassung



Abbildung 5.1: AI und iOS Zeitaufwand

Gesamt: 263,17h

Freizeit: 228,48h

Abbildungsverzeichnis

4.1	AI und iOS Zeitaufwand	38
5.1	AI und iOS Zeitaufwand	56

Code-Verweise

4.1	Auszug aus der Vite-Konfiguration	26
4.2	Routing-Struktur in App.jsx	27
4.3	Generische GET-Methode im API-Service	28
4.4	CSS für den Flip-Effekt	30
4.5	Theme-Switching mittels CSS-Variablen	32
4.6	Initialer State im QuizModal	33
4.7	Update-Logik für verschachtelte Antworten	33
4.8	Nutzung eines Sets fuer UI-State	34
4.9	Asynchrones Laden von Detaildaten	35
4.10	Dockerfile für das Frontend	36
5.1	Auszug aus der Middleware-Konfiguration	43
5.2	Verifizierung im AuthService	49